The file is an x64 GoLand Elf binary compiled for FreeBSD. Upon initial analysis the file appears to be a HTTP server that host a page on port 8080 that says ‘Nothing to see here :{’. Looking over the main.main function more thoroughly, we will notice a call to runtime.newproc, which is a GoLang method for creating threads. We will mostly be working with the assembly here, as most decompilers have issues with handling the many different calling conventions that various GoLang versions use.

```
00747e49    void** const var_70_1 = &data_2b33e8
00747e66    uint64_t* const var_70_2 = &net..stmp_87
00747e7b    void* const var_60 =
00747e7b        &go.itab.net/http.HandlerFunc,net/http.Handler
00747e87    void** const var_58 = &data_2b33e0
00747e8c    int64_t w = net/http.(*ServeMux).Handle(
00747e8c        mux: runtime.newproc(siz, fn),
```

```
00747e36  e8e51ce2ff        call   fmt.Fprintln
00747e3b  c7042400000000    mov    dword [rsp], 0x0
00747e42  488d059fb5b6ff    lea    rax, [rel data_2b33e8]
00747e49  4889442408        mov    qword [rsp+0x8 {var_70_1}], rax  {data_2b33e8}
00747e4e  e8cdb1daff        call   runtime.newproc
00747e53  90                nop
00747e54  488b05ed450000    mov    rax, qword [rel net/http.DefaultServeMux]
```

Looking at 0x2b33e8, we see that it points to a function named ‘net/http.init’

```
002b33e8  void* data_2b33e8 = net/http.init
```

And that function has some interesting PCap related calls. And again, we’ll need to look at the assembly as the decompilers miss a lot of code (like the call to rc4.NewCipher below)..

```
00748080    int64_t net/http.init()

00748092        int128_t* packet
00748092        void* fsbase
00748092
00748092        if (&packet u> *(*(fsbase - 8) + 0x10))
007480ba            int64_t var_d0_1 = 3
007480c3            int32_t var_c8_1 = 0x640
007480cb            char var_c4_1 = 1
007480d0            int64_t var_c0_1 = -0x989680
007480d9            struct github.com/google/gopacket/pcap.Handle* entry_handle
007480d9            time.Duration entry_timeout
007480d9            bool entry_promisc
007480d9            int32 entry_snaplen
007480d9            struct github.com/google/gopacket/pcap.Handle* p =
007480d9                github.com/google/gopacket/pcap.OpenLive(
007480d9                device: "re0resrfarfbrferidrisrjerlmrlprmcrmtrnarnmrpirrhrrprwprxesacsblscpsdosdrsdssdtse
007480d9                snaplen: entry_snaplen, promisc: entry_promisc, timeout: entry_timeout,
007480d9                handle: entry_handle, ~b4: 0x640)
007480e3            int e_1
007480e3            int e = e_1
007480e8            int64_t var_a8
007480e8            int64_t rdx = var_a8
007480e8
007480f3            if (e_1 == 0)
007481a             var_c8_1.q = 4
00748123            int64_t expr
00748123            github.com/google/gopacket/layers.LinkType rsi
00748123            struct github.com/google/gopacket/pcap.Handle* rdi
00748123            rsi, rdi =
00748123                github.com/google/gopacket/pcap.(*Handle).SetBPFFilter(p, expr, err: 4)
007486e6            int64_t expr_1 = expr
007486eb            e, rdx = runtime.gopanic(e: *-0xffffffffff676988)
007486f0            e = *(e + 8)
007480f3            else if (e != 0)
007486f0                e = *(e + 8)
007486f0
007486f8            int64_t var_d0_15 = rdx
00748700            runtime.gopanic(e)
00748700
? 00748706      runtime.morestack_noctxt()
00748706        noreturn
```

```
00748430  e86b71dbff        call   runtime.growslice
00748435  488b442428        mov    rax, qword [rsp+0x28 {e_1}]
0074843a  488b4c2438        mov    rcx, qword [rsp+0x38 {var_a0}]
0074843f  488b942480000000  mov    rdx, qword [rsp+0x80 {packet}]

00748447  0fb75c2444        movzx  ebx, word [rsp+0x44 {var_94_2}]
0074844c  6689580b          mov    word [rax+0xb], bx
00748450  90                nop
00748451  66817a263713      cmp    word [rdx+0x26], 0x1337
00748457  0f8593fdffff      jne    0x7481f0

0074845d  90                nop
0074845e  0fb75a10          movzx  ebx, word [rdx+0x10]
00748462  66c1c308          rol    bx, 0x8
00748466  6683fb20          cmp    bx, 0x20
0074846a  0f8580fdffff      jne    0x7481f0

00748470  90                nop
00748471  817a2ac6de5fe5    cmp    dword [rdx+0x2a], 0xe55fdec6
00748478  0f8572fdffff      jne    0x7481f0

0074847e  488b5c2448        mov    rbx, qword [rsp+0x48 {var_90_1}]
00748483  4883fb22          cmp    rbx, 0x22
00748487  0f8608020000      jbe    0x748695

0074848d  807a2208          cmp    byte [rdx+0x22], 0x8
00748491  0f8559fdffff      jne    0x7481f0

00748497  48890424          mov    qword [rsp {var_d8_7}], rax
0074849b  48c74424080d0000_ mov    qword [rsp+0x8 {var_d0_8}], 0xd
007484a4  48894c2410        mov    qword [rsp+0x10 {var_c8_1}], rcx
007484a9  e872c7edff        call   crypto/rc4.NewCipher
007484ae  488b442418        mov    rax, qword [rsp+0x18 {var_c0_1}]
007484b3  4889842488000000  mov    qword [rsp+0x88 {var_50_1}], rax
007484bb  488b4c2420        mov    rcx, qword [rsp+0x20 {var_b8}]
007484c0  488b542428        mov    rdx, qword [rsp+0x28 {e_1}]
007484c5  48837c242000      cmp    qword [rsp+0x20 {var_b8}], 0x0
007484cb  7449              je     0x748516

007484cd  450f57ff          xorps  xmm15, xmm15
007484d1  440f11bc24b00000_ movups xmmword [rsp+0xb0 {var_28}], xmm15
007484da  4885c9            test   rcx, rcx
```

We see two interesting function calls at the beginning of net/http.init, pcap.OpenLive and pcap.SetBPFFilter. Looking up the function definitions (https://pkg.go.dev/github.com/google/gopacket/pcap), we can see OpenLive expects a device (“re0” in this case for Realtek) and SetBPFFilter expects and expression and compiles and sets a Berkley Packet Filter (BPF) to capture and filter packets at the OS level. In this case ICMP packets.

The next important bit of code reads any packets that it captures.

```
007481cb  48890424          mov     qword [rsp {var_c8_4}], rax  {data_288289[0xa0], "icmpicm
007481cf  e80cf0e6ff        call    github.com/google/gopacket.(*PacketSource).Packets
007481d4  488b442408        mov     rax, qword [rsp+0x8 {var_c0}]  {data_288289[0xa0], "icmpi
007481d9  4889842498000000  mov     qword [rsp+0x98 {var_30}], rax  {data_288289[0xa0], "icmp
007481e1  450f57ff          xorps   xmm15, xmm15
007481e5  440f11bc24b00000… movups  xmmword [rsp+0xb0 {var_18}], xmm15
007481ee  eb08              jmp     0x7481f8

007481f0  488b842498000000  mov     rax, qword [rsp+0x98 {var_30}]

007481f8  48890424          mov     qword [rsp {var_c8_5}], rax
007481fc  488d8c24b0000000  lea     rcx, [rsp+0xb0 {var_18}]
00748204  48894c2408        mov     qword [rsp+0x8 {var_c0_2}], rcx {var_18}
00748209  e852d9d6ff        call    runtime.chanrecv2
0074820e  807c241000        cmp     byte [rsp+0x10 {var_b8_1}], 0x0
00748213  0f848c040000      je      0x7486a5

00748219  488b8424b0000000  mov     rax, qword [rsp+0xb0 {var_18}]
00748221  488b8c24b8000000  mov     rcx, qword [rsp+0xb8 {var_18+0x8}]
00748229  450f57ff          xorps   xmm15, xmm15
0074822d  440f11bc24b00000… movups  xmmword [rsp+0xb0 {var_18}], xmm15
00748236  488b4020          mov     rax, qword [rax+0x20]
0074823a  48890c24          mov     qword [rsp {var_c8_6}], rcx
0074823e  6690              nop
00748240  ffd0              call    rax
00748242  488b542418        mov     rdx, qword [rsp+0x18 {var_b0_1}]
00748247  488b442408        mov     rax, qword [rsp+0x8 {var_c0_2}]
0074824c  488b4c2410        mov     rcx, qword [rsp+0x10 {var_b8_1}]
00748251  4883fa18          cmp     rdx, 0x18
00748255  0f828a040000      jb      0x7486e5

0074825b  0f1f440000        nop     dword [rax+rax]
00748260  4883fa26          cmp     rdx, 0x26
00748264  0f826f040000      jb      0x7486d9

0074826a  4883fa28          cmp     rdx, 0x28
0074826e  0f825b040000      jb      0x7486cf

00748274  4883fa2e          cmp     rdx, 0x2e
00748278  0f8247040000      jb      0x7486c5
```

By following the return values of the call to RAX at 0x748240, we can deduce that RDX is likely the error value, RCX the size, and RAX the captured packet.

After receiving a packet, it will modify and store some bytes in a separate variable. It starts reading four bytes from the packet at offset 0x2a and two bytes from offset 0x24. It then xors the two bytes with each 2 bytes of the dword and stores the values.

```
0074828a  8b482a            mov     ecx, dword [rax+0x2a]
0074828d  894c2444          mov     dword [rsp+0x44 {_0x2a}], ecx
00748291  0fb75024          movzx   edx, word [rax+0x24]
00748295  6689542442        mov     word [rsp+0x42 {_0x24}], dx
0074829a  488d1d7fd4adff    lea     rbx, [rel data_225720]
007482a1  48891c24          mov     qword [rsp {var_c8_7}], rbx  {data_225720}
007482a5  48c7442408020000… mov     qword [rsp+0x8 {var_c0}], 0x2
007482ae  48c7442410020000… mov     qword [rsp+0x10 {packetLen}], 0x2
007482b7  e8c471dbff        call    runtime.makeslice
007482bc  488b442418        mov     rax, qword [rsp+0x18 {error}]
007482c1  66c74424400000    mov     word [rsp+0x40 {var_88_1}], 0x0
007482c8  8b4c2444          mov     ecx, dword [rsp+0x44 {_0x2a}]
007482cc  89ca              mov     edx, ecx
007482ce  c1e910            shr     ecx, 0x10
007482d1  0fb75c2442        movzx   ebx, word [rsp+0x42 {_0x24}]
007482d6  31d9              xor     ecx, ebx
007482d8  90                nop
007482d9  66c1c108          rol     cx, 0x8
007482dd  668908            mov     word [rax], cx
007482e0  31d3              xor     ebx, edx
007482e2  90                nop
007482e3  66c1c308          rol     bx, 0x8
007482e7  66895c2440        mov     word [rsp+0x40 {_0x2a_lower16bits}], bx
007482ec  488d0d2dd4adff    lea     rcx, [rel data_225720]
007482f3  48890c24          mov     qword [rsp {var_c8}], rcx  {data_225720}
007482f7  4889442408        mov     qword [rsp+0x8 {var_c0_3}], rax
007482fc  48c7442410020000… mov     qword [rsp+0x10 {var_b8}], 0x2
00748305  48c7442418020000… mov     qword [rsp+0x18 {var_b0}], 0x2
0074830e  48c7442420060000… mov     qword [rsp+0x20 {expr}], 0x6
00748317  e88472dbff        call    runtime.growslice
```

It begins storing bytes from various offsets in the packet to a new variable, starting with the first two xored values. The next four bytes are in our packet at offset 0x14, followed by two bytes as 0x24, and four bytes at 0x2a.

```
0074831c  488b442428          mov     rax, qword [rsp+0x28 {key}]
00748321  488b4c2438          mov     rcx, qword [rsp+0x38 {var_90}]
00748326  488b542470          mov     rdx, qword [rsp+0x70 {packet_1}]
0074832b  8b5a14              mov     ebx, dword [rdx+0x14]
0074832e  895802              mov     dword [rax+0x2], ebx
00748331  4883f908            cmp     rcx, 0x8
00748335  733d                jae     0x748374

00748337  488d15e2d3adff      lea     rdx, [rel data_225720]
0074833e  48891424            mov     qword [rsp {var_c8}], rdx  {data_225720}
00748342  4889442408          mov     qword [rsp+0x8 {var_c0_3}], rax
00748347  48c7442410060000…   mov     qword [rsp+0x10 {packetLen}], 0x6
00748350  48894c2418          mov     qword [rsp+0x18 {error}], rcx
00748355  48c7442420080000…   mov     qword [rsp+0x20 {var_a8}], 0x8
0074835e  6690                nop
00748360  e83b72dbff          call    runtime.growslice
00748365  488b442428          mov     rax, qword [rsp+0x28 {key}]
0074836a  488b4c2438          mov     rcx, qword [rsp+0x38 {var_90}]
0074836f  488b542470          mov     rdx, qword [rsp+0x70 {packet_1}]

00748374  0fb75a24            movzx   ebx, word [rdx+0x24]
00748378  66895806            mov     word [rax+0x6], bx
0074837c  0f1f4000            nop     dword [rax]
00748380  4883f90c            cmp     rcx, 0xc
00748384  733b                jae     0x7483c1

00748386  488d1593d3adff      lea     rdx, [rel data_225720]
0074838d  48891424            mov     qword [rsp {var_c8}], rdx  {data_225720}
00748391  4889442408          mov     qword [rsp+0x8 {var_c0_3}], rax
00748396  48c7442410080000…   mov     qword [rsp+0x10 {packetLen}], 0x8
0074839f  48894c2418          mov     qword [rsp+0x18 {error}], rcx
007483a4  48c74424200c0000…   mov     qword [rsp+0x20 {var_a8}], 0xc
007483ad  e8ee71dbff          call    runtime.growslice
007483b2  488b442428          mov     rax, qword [rsp+0x28 {key}]
007483b7  488b4c2438          mov     rcx, qword [rsp+0x38 {var_90}]
007483bc  488b542470          mov     rdx, qword [rsp+0x70 {packet_1}]

007483c1  8b5a2a              mov     ebx, dword [rdx+0x2a]
007483c4  895808              mov     dword [rax+0x8], ebx
```

The final four bytes stored from the packet are the word at offset 0x26 and the other two bytes that were xored earlier.

```
00748408  0fb75a26          movzx   ebx, word [rdx+0x26]
0074840c  6689580c          mov     word [rax+0xc], bx
00748410  4883f910          cmp     rcx, 0x10
00748414  733e              jae     0x748454

00748416  488d1503d3adff    lea     rdx, [rel data_225720]
0074841d  48891424          mov     qword [rsp {var_c8}], rdx  {data_225720}
00748421  4889442408        mov     qword [rsp+0x8 {var_c0_3}], rax
00748426  48c74424100e0000… mov     qword [rsp+0x10 {packetLen}], 0xe
0074842f  48894c2418        mov     qword [rsp+0x18 {error}], rcx
00748434  48c7442420100000… mov     qword [rsp+0x20 {var_a8}], 0x10
0074843d  0f1f00            nop     dword [rax]
00748440  e85b71dbff        call    runtime.growslice
00748445  488b442428        mov     rax, qword [rsp+0x28 {key}]
0074844a  488b4c2438        mov     rcx, qword [rsp+0x38 {var_90}]
0074844f  488b542470        mov     rdx, qword [rsp+0x70 {packet_1}]

00748454  0fb75c2440        movzx   ebx, word [rsp+0x40 {_0x2a_lower16bits}]
00748459  6689580e          mov     word [rax+0xe], bx
```

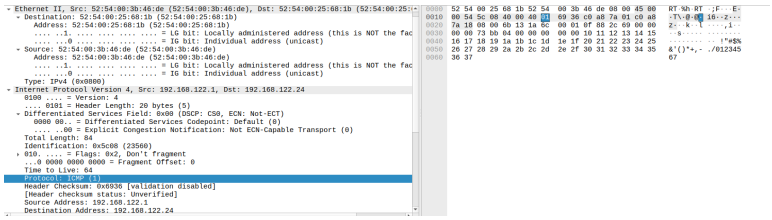
Next it will check the values of some of these bytes, it will check that offset ox26 in the packet is equal to 0x1337, offset 0x10 is 0x20, and offset 0x2a is 0xe55fdec6, it also checks that offset 0x22 is 8.

```
0074845e  66817a263713      cmp     word [rdx+0x26], 0x1337
00748464  0f8586fdffff      jne     0x7481f0

0074846a  90                nop
0074846b  0fb75a10          movzx   ebx, word [rdx+0x10]
0074846f  66c1c308          rol     bx, 0x8
00748473  6683fb20          cmp     bx, 0x20
00748477  0f8573fdffff      jne     0x7481f0

0074847d  90                nop
0074847e  817a2ac6de5fe5    cmp     dword [rdx+0x2a], 0xe55fdec6
00748485  0f8565fdffff      jne     0x7481f0

0074848b  488b5c2450        mov     rbx, qword [rsp+0x50 {packetLen_1}]
00748490  4883fb22          cmp     rbx, 0x22
00748494  0f861b020000      jbe     0x7486b5

0074849a  807a2208          cmp     byte [rdx+0x22], 0x8
0074849e  6690              nop
007484a0  0f854afdffff      jne     0x7481f0
```

Finally, the bytes stored in the variable composed of various bytes from the packet are used as the key to AES CBC decrypt the flag, notice the key is also used as the IV in the call to NewCBCDecrypter

```
007484d2  48b951f1a529b4df…  mov     rcx, 0xc07edfb429a5f151
007484dc  488908             mov     qword [rax], rcx  {-0x3f81204bd65a0eaf}
007484df  48b92a3b2f8f243d…  mov     rcx, 0xb34e3d248f2f3b2a
007484e9  48894808           mov     qword [rax+0x8], rcx  {-0x4cb1c2db70d0c4d6}
007484ed  48b95aedb0cf0b9c…  mov     rcx, 0x8cdd9c0bcfb0ed5a
007484f7  48894810           mov     qword [rax+0x10], rcx  {-0x732263f4304f12a6}
007484fb  48b9cde60e9b3ec4…  mov     rcx, 0xc64c43e9b0ee6cd
00748505  48894818           mov     qword [rax+0x18], rcx  {0xc64c43e9b0ee6cd}
00748509  488b4c2460         mov     rcx, qword [rsp+0x60 {key_1}]
0074850e  48890c24           mov     qword [rsp {var_c8_9}], rcx
00748512  48c7442408100000…  mov     qword [rsp+0x8 {var_c0_4}], 0x10
0074851b  488b542448         mov     rdx, qword [rsp+0x48 {var_80_1}]
00748520  4889542410         mov     qword [rsp+0x10 {packetLen}], rdx
00748525  e8d608eeff         call    crypto/aes.NewCipher
0074852a  488b442418         mov     rax, qword [rsp+0x18 {error}]
0074852f  4889842488000000   mov     qword [rsp+0x88 {var_40_1}], rax
00748537  488b4c2420         mov     rcx, qword [rsp+0x20 {var_a8}]
0074853c  48894c2478         mov     qword [rsp+0x78 {var_50_1}], rcx
00748541  488d15d8d1adff     lea     rdx, [rel data_225720]
00748548  48891424           mov     qword [rsp {var_c8_10}], rdx  {data_225720}
0074854c  48c7442408200000…  mov     qword [rsp+0x8 {var_c0}], 0x20
00748555  48c7442410200000…  mov     qword [rsp+0x10 {var_b8}], 0x20
0074855e  6690               nop
00748560  e81b6fdbff         call    runtime.makeslice
00748565  488b442418         mov     rax, qword [rsp+0x18 {error}]
0074856a  4889842480000000   mov     qword [rsp+0x80 {var_48}], rax
00748572  488b8c2488000000   mov     rcx, qword [rsp+0x88 {var_40_1}]
0074857a  48890c24           mov     qword [rsp {var_c8_11}], rcx
0074857e  488b4c2478         mov     rcx, qword [rsp+0x78 {var_50_1}]
00748583  48894c2408         mov     qword [rsp+0x8 {var_c0_6}], rcx
00748588  488b4c2460         mov     rcx, qword [rsp+0x60 {key_1}]
0074858d  48894c2410         mov     qword [rsp+0x10 {packetLen}], rcx
00748592  48c7442418100000…  mov     qword [rsp+0x18 {var_b0}], 0x10
0074859b  488b4c2448         mov     rcx, qword [rsp+0x48 {var_80_1}]
007485a0  48894c2420         mov     qword [rsp+0x20 {var_a8_1}], rcx
007485a5  e8b6ccedff         call    crypto/cipher.NewCBCDecrypter
```

So we have seven bytes of the sixteen byte AES key. To figure out the rest of the key, we will need to take a look at the structure of IP (https://en.wikipedia.org/wiki/IPv4#Packet_structure) and ICMP (https://en.wikipedia.org/wiki/Internet_Control_Message_Protocol) packet to see what is expected in our key.
And a PCAP from a ping will help find the expected values.



Key offset	Packet offset	Field	Size	Expected value
0x0	0x24 xored 0x2a (high bytes)		2	
0x2	0x14	IP - Flags, Fragments, TTL, Protocol	4	0x40004001
0x6	0x24	ICMP - Checksum	2	
0x8	0x26	ICMP - ID	2	0x1337
0xc	0x2a	ICMP - Timestamp	4	0xe55fdec6
0xe	0x24 xored 0x2a (low bytes)		2	

Now we have most of the information we need to solve the problem, we just need the compute the checksum to obtain the final bytes of the key. The checksum is easy enough to compute (https://www.ietf.org/rfc/rfc1071.txt) since we have all the fields in the ICMP packet needed to compute it. This python function will compute the checksum for us:

```
def calculate_checksum(data):
    checksum = 0
    if len(data) % 2 != 0:
        data += b"\x00"
    for i in range(0, len(data), 2):
        checksum += (data[i] << 8) + data[i+1]
    checksum = (checksum >> 16)+ (checksum & 0xffff)
    checksum += checksum >> 16
    return (~checksum) & 0xffff
```

Method 1 Solution:Static

The RFC and ICMP Wikipedia article linked above are helpful in building the ICMP packet used to compute our checksum.

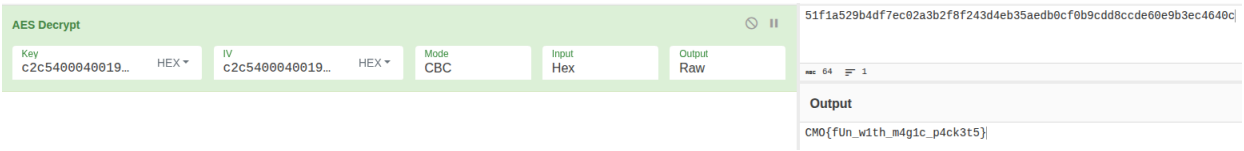
```
        Type | Code | Checksum | ICMP Id | Seq #  | Data
data  =  08      00 +      0000 +      3713   + 0001  |  c6de +  f5e5
```

data = b’\x08\x00\x00\x00\x37\x13\x00\x01\xc6\xde\5f\e5’

Which when ran through our caclulate_checksum function returns 0x9a27 and gives us all we need to construct our key:
c2c5400040019a27c6de5fe53713f95c

The encrypted flag starts being build at 0x7484d2 and we can reconstruct it from the code:
51f1a529b4df7ec02a3b2f8f243d4eb35aedb0cf0b9cdd8ccde60e9b3ec4640c

And now we can use something like CyberChef to decrypt the flag for us:



CMO{fUn_w1th_m4g1c_p4ck3t5}

Method 2: Dynamic
Modifying python code found on the internet (https://denizhalil.com/2024/04/06/sending-icmp-packets-with-python-socket-adventure-in-signaling/), we can create the custom the ICMP packet and send it to a host that has the httpd binary running. Heres my python script, it uses the above compute_checksum function:

```
def send_icmp_packet(target_ip, target_port=0, data=b'\xc6\xde\x5f\xe5', ttl=64, icmp_id=0x3713):
        icmp_type = 8  # ICMP echo request
        icmp_code = 0
        icmp_checksum = 0
        icmp_sequence = 1
        # ICMP payload (data)
        icmp_payload = data
        # ICMP header
        icmp_header = struct.pack("!BBHHH", icmp_type, icmp_code, icmp_checksum, icmp_id, icmp_sequence)
        # Calculate checksum
        icmp_checksum = calculate_checksum(icmp_header + icmp_payload)
        # Update ICMP header with correct checksum

        icmp_header = struct.pack("!BBHHH", icmp_type, icmp_code, (icmp_checksum), icmp_id, icmp_sequence)

        # Create ICMP packet
        icmp_packet = icmp_header + icmp_payload

        # Create raw socket
```

```
    with socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROTO_ICMP) as sock:
        # Set socket TTL
        sock.setsockopt(socket.IPPROTO_IP, socket.IP_TTL, struct.pack("I", ttl))

        # Set socket timeout
        sock.settimeout(2)

        # Send packet
        sock.sendto(icmp_packet, (target_ip, target_port))

    print("ICMP packet sent successfully!")
    return
```

```
crudd@free14:~/httpd3 $ sudo ./httpd
Starting server...
Server is running at http://localhost:8080
CMO{fUn_w1th_m4g1c_p4ck3t5}
```

```
In [10]: send_icmp_packet('192.168.122.24', 0)
ICMP packet sent successfully!

In [11]:
```